

MOHLOLI ISA SPECIFICATION

For Low-Cost AI-Enabled Mobile Phones

Document Version: 1.0

Date: October 2025

1. OVERVIEW & MOTIVATION

1.1 Purpose and Application Context

The Mohloli ISA (named from Sesotho "The Inventor") serves as the computational foundation for affordable mobile devices targeting African markets. It is specifically engineered to efficiently run localized AI applications including:

- Voice recognition in indigenous languages
- Biometric authentication for mobile financial services
- Lightweight health monitoring tools
- Educational AI applications

1.2 Design Goals

- Computational Efficiency: Optimized for AI workload patterns
- Power Economy: Minimal energy consumption for extended battery life
- Hardware Simplicity: Low-cost processor implementation
- Local Relevance: Tailored to African mobile usage patterns

1.3 Target Workload Suitability

The ISA directly addresses computational demands identified in domain analysis, providing native support for floating-point operations and custom instructions that accelerate core AI processing tasks.

2. ARCHITECTURAL DESIGN CHOICES

2.1 Instruction Set Philosophy

- Load-Store RISC architecture prioritizing:
- Simple, fixed-length instruction set
- Streamlined processor pipeline
- Reduced power consumption
- Low design complexity

2.2 Register File Specification

Number	Name	Roles
x0	Zero	Hardwired Zero value
x1	lr	Link Register(return addresses)
x2	sp	Stack Pointer
x3-x31	-	General Purpose

Total: 32 general-purpose registers, 32-bit width

2.3 Supported Data Types

- Integers: 8-bit (byte), 16-bit (half-word), 32-bit (word)
- Floating-Point: 32-bit single-precision (IEEE 754)

2.4 Addressing Modes

1. Register Direct: op dest, reg1, reg2
2. Immediate: op dest, reg1, #constant
3. Base + Offset: load dest, #offset(reg1)

2.5 Memory Model

- Byte Ordering: Little-endian
- Alignment: Word (4-byte) and half-word (2-byte) alignment enforced

2.6 Instruction Format

- Fixed-length 32-bit instructions enabling simple and fast decoding.

3. INSTRUCTION SET SUMMARY

3.1 Integer Arithmetic Instructions

Mnemonic	Operands	Description
plus	rd, rs1, rs2	Integer addition
plusi	rd, rs1, imm12	Integer add immediate
min	rd, rs1, rs2	Integer subtraction
mult	rd, rs1, rs2	Integer multiplication

3.2 Floating Point Arithmetic Instructions

Mnemonic	Operands	Description
fplus	rd, rs1, rs2	Floating-point addition
fmin	rd, rs1, rs2	Floating-point subtraction
fmult	rd, rs1, rs2	Floating-point multiplication

3.3 Memory Address Instruction

Mnemonic	Operands	Description
load	rd, imm12(rs1)	Load word from memory
store	rs2, imm12(rs1)	Store word to memory
fload	rd, imm12(rs1)	Load float from memory
fstore	rs2, imm12(rs1)	Store float to memory

3.4 Control Flow Instructions

Mnemonic	Operands	Description
jumpL	rd, imm20	Jump and link (function calls)
branchEQ	rs1, rs2, imm12	Branch if equal
branchNE	rs1, rs2, imm12	Branch if not equal

3.5 Custom Extensions Instructions

Mnemonic	Operands	Description	Target Workload
vecMAC	rd, rs1, rs2	Vector multiply-accumulate	Voice/Biometrics
vecSubSq	rd, rs1, rs2	Vector subtract & square	Voice/Biometrics
branchGT	rs1, rs2, imm12	Branch if greater than	Health/MFS

4. Instructions Encoding

4.1 Standard R-Type Format

Bits 31-25: funct7

Bits 24-20: rs2

Bits 19-15: rs1

Bits 14-12: funct3

Bits 11-7: rd

Bits 6-0: opcode

4.2 Custom Instruction Encoding

Custom Opcode: 1101011

Operation Mapping:

vecMAC: funct7 = 0000001

vecSubSq: funct7 = 0000010

branchGT: funct7 = 0000011

5. DESIGN RATIONALE & TRADE-OFFS

5.1 Simplicity vs. Capability

Choice: Minimal base instruction set with strategic custom extensions

Rationale: Reduces control logic complexity while providing significant acceleration for key AI workloads through targeted custom instructions.

5.2 Code Density vs. Performance

Choice: Fixed 32-bit instruction length

Rationale: Enables simple decoding and high performance, with provision for future compressed instruction extension.

5.3 Hardware Implementation Impact

Choice: Custom instructions that leverage existing arithmetic units

Rationale: Maximizes performance gain for AI workloads while minimizing additional hardware complexity.

5.4 Extensibility

Choice: Dedicated custom opcode space with ample funct7 range

Rationale: Provides clear pathway for future instruction additions as application requirements evolve.

6. CONCLUSION

The Mohloli ISA represents a balanced approach to processor design for the African mobile context. By combining RISC simplicity with targeted custom extensions, it delivers the computational efficiency needed for AI applications while maintaining the power economy and cost-effectiveness required for widespread adoption.